```
  def change
    validate_check_constraint :trips, name: "rating_check"
  end
end
```

In the first Migration, you'll see that Strong Migrations prints that it detected a potentially dangerous migration.

Remember that constraints like CHECK may validate all rows or only new roes. Adding a CHECK constraint to a table requires an Exclusive Access lock while being added. Normally this is quick, but when all rows need to be validated, this extends the length of the lock. As you saw earlier, with a lock in place even for ten seconds many queries in your application might be blocked, causing application errors.

Instead, Strong Migrations suggests using "validate: false" in your Migration, which means the constraint is added without validating all rows; only new rows will be validated when they are created. This means the exclusive lock is very brief causing little or no interruption.

But what about the existing rows that aren't being modified, that existed before the constraint was added?

Those rows have not yet been validated for compatibility with the CHECKJ constraint condition. This is not a good final state to be in, because you're not able to validate all table data.

To do this you'll need to validate the constraint. This is accomplished with a second Migration that calls the validate_check_constraint method.

Prior to running this, in psql run \dt trips and confirms the constraint is visible but has the keywords NOT VALID:

```
Check constraints:
    "rating_check" CHECK (rating >= 1 AND rating <= 5) NOT VALID
```

With the second migration, you'll validate the constraint. After running it you should no longer see the NOT VALID keywords.

With a validated constraint, you can be confident that all values for the rating match the CHECK constraint definition.

Locking, Blocking, and Concurrency Refresher

As described earlier, one of the operational challenges with PostgreSQL is when tables are locked for modifications and queries get blocked.

This section will go into a bit more depth on that.